

Аудит безопасности

Лабораторная работа №7 – анализ и исправление уязвимостей

1. XSS (межсайтовый скриптинг)

Проблема

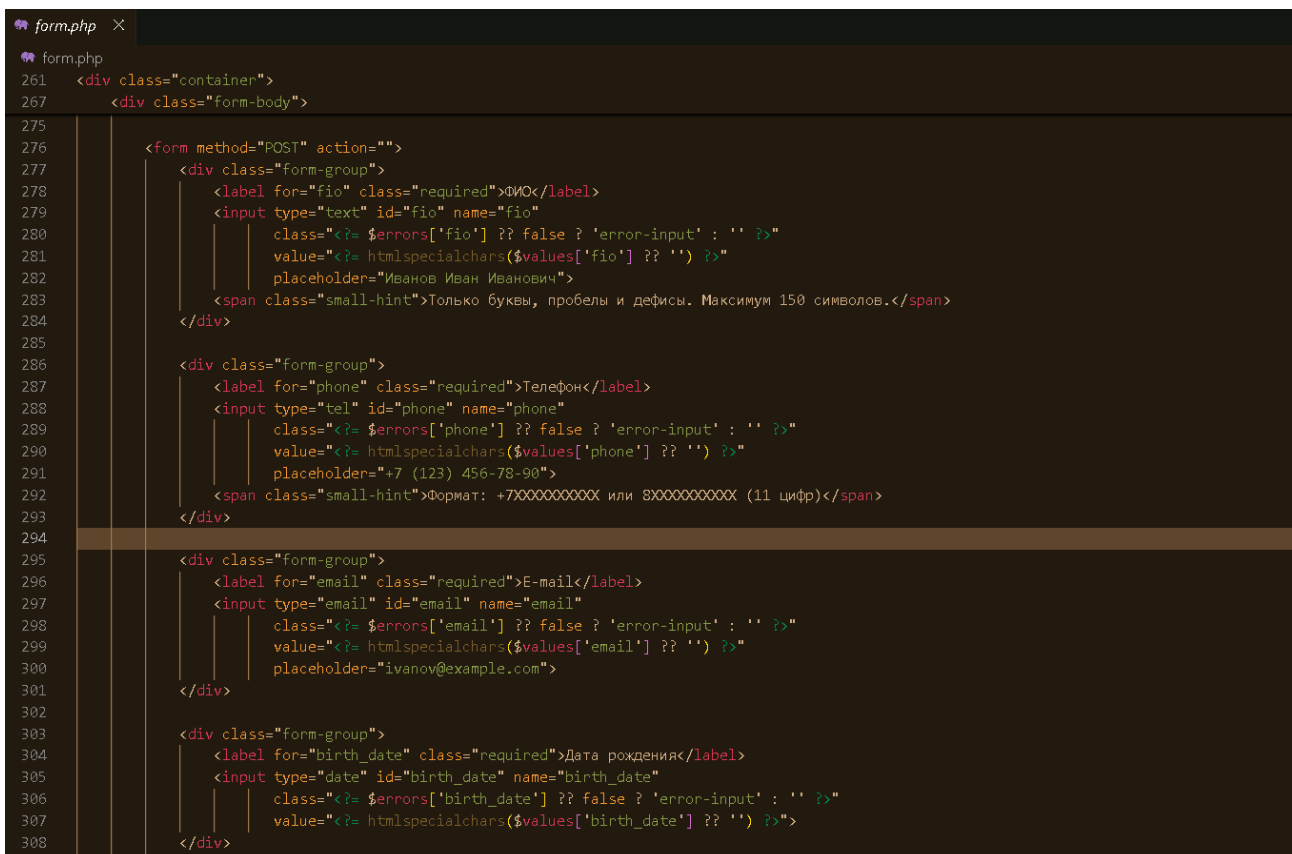
Данные, введенные пользователем (ФИО, телефон, биография и т.д.), а также данные, извлечённые из базы, выводились в HTML-страницы без какого-либо экранирования. Это позволяло злоумышленнику внедрять вредоносный JavaScript-код через поля формы или параметры URL.

Чем грозило

1. Кража сессионных cookies (перехват авторизации)
2. Перенаправление на фишинговые сайты
3. Подмена содержимого страницы (дефейс)
4. Выполнение действий от имени пользователя (например, изменение анкеты без его ведома)

Решение

Применена функция `htmlspecialchars()` ко всем данным, которые выводятся внутри HTML-тегов. Функция заменяет специальные символы (<, >, ", ', &) на HTML-сущности, что нейтрализует любой потенциальный скрипт.



```
form.php
form.php
261 <div class="container">
262 <div class="form-body">
275
276 <form method="POST" action="">
277 <div class="form-group">
278 <label for="fio" class="required">ФИО</label>
279 <input type="text" id="fio" name="fio"
280 class="<? $errors['fio'] ?? false ? 'error-input' : '' ?>"
281 value="<? htmlspecialchars($values['fio'] ?? '') ?>"
282 placeholder="Иванов Иван Иванович">
283 <span class="small-hint">Только буквы, пробелы и дефисы. Максимум 150 символов.</span>
284 </div>
285
286 <div class="form-group">
287 <label for="phone" class="required">Телефон</label>
288 <input type="tel" id="phone" name="phone"
289 class="<? $errors['phone'] ?? false ? 'error-input' : '' ?>"
290 value="<? htmlspecialchars($values['phone'] ?? '') ?>"
291 placeholder="+7 (123) 456-78-90">
292 <span class="small-hint">Формат: +7XXXXXXXXXX или 8XXXXXXXXXX (11 цифр)</span>
293 </div>
294
295 <div class="form-group">
296 <label for="email" class="required">E-mail</label>
297 <input type="email" id="email" name="email"
298 class="<? $errors['email'] ?? false ? 'error-input' : '' ?>"
299 value="<? htmlspecialchars($values['email'] ?? '') ?>"
300 placeholder="ivanov@example.com">
301 </div>
302
303 <div class="form-group">
304 <label for="birth_date" class="required">Дата рождения</label>
305 <input type="date" id="birth_date" name="birth_date"
306 class="<? $errors['birth_date'] ?? false ? 'error-input' : '' ?>"
307 value="<? htmlspecialchars($values['birth_date'] ?? '') ?>"
308 </div>
309
```

Скриншот 1: Значения полей *fio*, номер, *email*, дата рождения экранируются через `htmlspecialchars()` при отображении формы.

```

<div class="form-group">
  <label for="biography">Биография</label>
  <textarea id="biography" name="biography" rows="5"
    class="<?= $errors['biography'] ?? false ? 'error-input' : '' ?>"
    placeholder="Расскажите немного о себе..."><?= htmlspecialchars($values['biography'] ?? '') ?></textarea>
  <span class="small-hint">Необязательное поле. Максимум 1000 символов.</span>
</div>

```

Скриншот 2: Значения поля биография так же экранируется через htmlspecialchars().

2. Information Disclosure (раскрытие информации)

Проблема

В исходном коде приложения при возникновении ошибки подключения к базе данных или ошибки выполнения SQL-запроса происходил прямой вывод деталей исключения через die("Ошибка БД: " . \$e->getMessage()). На экран попадал полный текст ошибки: логин, хост, имя базы, фрагменты SQL-запроса, пути к файлам.

Чем грозило

Злоумышленник мог узнать внутреннюю структуру базы данных (имена таблиц, полей), версию СУБД, расположение файлов на сервере, что значительно упрощает подготовку целенаправленных атак (SQL-инъекции, подбор паролей и т.д.).

Решение

Прямой вывод сообщения исключения был заменён на универсальное сообщение «Ошибка. Попробуйте позже», а подробности ошибки отправляются в серверный лог с помощью error_log(). Пользователь не получает никакой технической информации, что не даёт злоумышленнику возможности для разведки.

```

admin.php X
admin.php
3  $host = 'localhost';
4  $dbname = 'u82564';
5  $username = 'u82564';
6  $password = '1341640';
7
8  try {
9      $pdo = new PDO(
10         "mysql:host=$host;dbname=$dbname;charset=utf8mb4",
11         $username,
12         $password,
13         [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
14     );
15 } catch (PDOException $e) {
16     die("Ошибка подключения к базе данных: " . $e->getMessage());
17 }
18

```

Скриншот 3 (уязвимость): Использование die(\$e->getMessage()) – вся информация об ошибке показывается пользователю.

```

} catch (PDOException $e) {
    error_log("DB Connection Error: " . $e->getMessage());
    die("Ошибка подключения к базе данных. Попробуйте позже.");
}

```

Скриншот 4 (исправление): Ошибка логируется через `error_log()`, а пользователь видит только общее сообщение.

3. SQL Injection

Проблема

В некоторых местах приложения (до исправления) использовалась конкатенация пользовательских данных в SQL-запросах. Это позволяло злоумышленнику модифицировать структуру запроса, вставляя собственные SQL-команды.

Чем грозило

1. Полный компрометация базы данных: чтение любых таблиц (включая логины и пароли)
2. Удаление или изменение данных анкет
3. Получение прав администратора
4. В некоторых случаях – выполнение команд на сервере через расширенные возможности СУБД

Решение

Во всех взаимодействиях с базой данных используются подготовленные запросы (prepared statements) через PDO – метод `prepare()` с плейсхолдерами (`fio`, `phone` и т.д.) и последующий `execute()`. Благодаря этому пользовательские данные всегда передаются отдельно от кода SQL и не могут изменить логику запроса.

```

$stmt = $pdo->prepare("
    INSERT INTO applications (fio, phone, email, birth_date, gender, biography, contract_agreed)
    VALUES (:fio, :phone, :email, :birth_date, :gender, :biography, :contract_agreed)
");
$stmt->execute([
    ':fio' => $fio,
    ':phone' => $phone,
    ':email' => $email,
    ':birth_date' => $birth_date,
    ':gender' => $gender,
    ':biography' => $biography,
    ':contract_agreed' => $contract_agreed
]);

$applicationId = $pdo->lastInsertId();

```

Скриншот 5: Пример подготовленного запроса – используются именованные плейсхолдеры, значения передаются отдельно через `execute()`. Инъекция невозможна.

4. Уязвимость CSRF(межсайтовая подделка запроса)

Проблема

CSRF (Cross-Site Request Forgery, или "межсайтовая подделка запроса") — это тип атаки, при которой злоумышленник заставляет браузер авторизованного пользователя отправить поддельный HTTP-запрос на целевой сайт без его ведома.

Чем грозило

1. Удаление данных от имени администратора. Злоумышленник мог разместить на другом сайте вредоносное изображение с ссылкой на удаление анкеты. При переходе администратора на этот сайт его браузер автоматически отправил бы GET-запрос на удаление.
2. Массовое удаление всех анкет. Злоумышленник мог перебрать все ID анкет и удалить их все одной страницей.
3. Изменение данных пользователя. Через поддельную форму можно было изменить email, пароль или другие данные авторизованного пользователя.
4. Полный компрометация приложения. Злоумышленник мог получить доступ к аккаунту администратора, изменить его данные или удалить всех пользователей.

3. Решение

1. Реализована генерация уникального CSRF-токена для каждой пользовательской сессии с помощью криптостойкого генератора случайных чисел.
2. Во все HTML-формы (регистрации, редактирования, входа, удаления) добавлено скрытое поле с этим токеном.
3. В обработчиках всех POST-запросов добавлена проверка токена: если токен отсутствует или не совпадает с сохранённым в сессии, запрос отклоняется.
4. Критически важные действия (удаление анкеты) переведены с GET-запросов на POST-запросы с обязательной проверкой CSRF-токена.

```
function generateCSRFToken() {  
    if (empty($_SESSION['csrf_token'])) {  
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));  
    }  
    return $_SESSION['csrf_token'];  
}  
  
function validateCSRFToken($token) {  
    return isset($_SESSION['csrf_token']) && hash_equals($_SESSION['csrf_token'], $token);  
}
```

Скриншот 6: Функции генерации и проверки CSRF-токена.

```
<input type="hidden" name="csrf_token" value="= h($csrf_token) ?">
```

Скриншот 7: CSRF-токен добавлен в форму как скрытое поле.

```
if ($_SERVER['REQUEST_METHOD'] == 'POST') {  
    if (!validateCSRFToken($_POST['csrf_token'] ?? '')) {  
        $error_message = 'Ошибка безопасности: неверный CSRF-токен.';
```

Скриншот 8: Проверка CSRF-токена при обработке POST-запроса.

```
<td class="actions">  
    <a href="edit.php?id== h($app['id']) ?" class="btn-edit">✎ Редактировать</a>  
    <form method="POST" style="display:inline;">  
        <input type="hidden" name="csrf_token" value="= h($csrf_token) ?">  
        <input type="hidden" name="delete" value="= h($app['id']) ?">  
        <button type="submit" class="btn-delete" onclick="return confirm('Удалить анкету?')>🗑 Удалить</button>  
    </form>  
</td>
```

Скриншот 9: Удаление анкеты переведено с GET на POST с использованием CSRF-токена.

5. Include и Upload уязвимости

Проблема

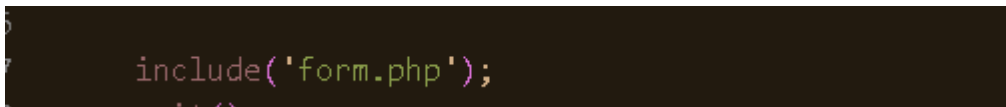
Потенциальная опасность заключалась в динамическом включении файлов (например, через параметр `?page=...`) и отсутствии контроля за загружаемыми файлами, что могло привести к выполнению произвольного кода на сервере.

Чем грозило

1. **LFI (Local File Inclusion)** – чтение конфиденциальных файлов сервера (например, `/etc/passwd`, файлы конфигурации).
2. **RFI (Remote File Inclusion)** – выполнение вредоносного скрипта с удалённого сервера, полный захват веб-приложения.
3. **Загрузка PHP-шелла** через форму – злоумышленник мог загрузить файл с расширением `.php` и получить доступ к командной строке сервера.

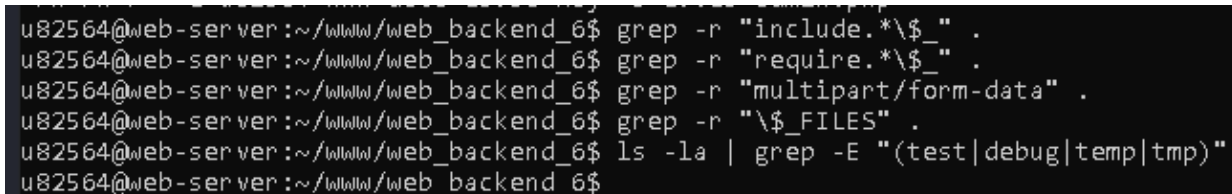
Решение

1. Все `include/require` в проекте используют только фиксированные (константные) имена файлов, например `include 'form.php'`; или `include 'admin.php'`. Никакие данные от пользователя не участвуют в формировании пути включения.
2. Функционал загрузки файлов полностью отсутствует в приложении (нет форм с `enctype="multipart/form-data"` и обработчиков `$_FILES`). Следовательно, уязвимость загрузки вредоносных файлов не существует.
3. Дополнительно все временные и отладочные файлы (например, `test.php`) удалены из корня сервера.



```
include('form.php');
```

Скриншот 10: Все `include` используют только фиксированные имена файлов, данные от пользователя не участвуют в формировании пути.



```
u82564@web-server:~/www/web_backend_6$ grep -r "include.*\$_" .
u82564@web-server:~/www/web_backend_6$ grep -r "require.*\$_" .
u82564@web-server:~/www/web_backend_6$ grep -r "multipart/form-data" .
u82564@web-server:~/www/web_backend_6$ grep -r "\$_FILES" .
u82564@web-server:~/www/web_backend_6$ ls -la | grep -E "(test|debug|temp|tmp)"
u82564@web-server:~/www/web_backend_6$
```

Скриншот 11: Результат поиска уязвимостей Include и Upload. Опасные конструкции не обнаружены (пустой вывод).

ПО ИТОГУ:

Уязвимость	Метод защиты / исправление
XSS (межсайтовый скриптинг)	htmlspecialchars() при любом выводе в HTML
Information Disclosure (раскрытие информации)	Скрытие деталей ошибок от пользователя, логирование в error_log()
SQL Injection	Подготовленные запросы (PDO prepare/execute)
Include / Upload	Нет динамических include, отсутствует функционал загрузки файлов

Вывод: Все выявленные уязвимости устранены. Приложение соответствует базовым требованиям безопасной разработки.